

```
1 class NaiveBayes:
2     def __init__(self):
3         self.prior = {}
4         self.conditional = {}
5         self.classes = []
6     def fit(self, data):
7         total_records = len(data)
8         labels = [row[-1] for row in data]
9         self.classes = list(set(labels))
10
11     # Calculate Priors
12     for c in self.classes:
13         self.prior[c] = labels.count(c) / total_records
14         self.conditional[c] = {}
15
16     # Calculate Conditionals
17     num_features = len(data[0]) - 1
18     for c in self.classes:
19         sub_data = [row[:-1] for row in data if row[-1] == c]
20         sub_total = len(sub_data)
21
22         for f_idx in range(num_features):
23             self.conditional[c][f_idx] = {}
24             feature_vals = [row[f_idx] for row in data]
```

```
23     self.conditional[c][f_idx] = {}
24     feature_vals = [row[f_idx] for row in data]
25     for val in set(feature_vals):
26         count = sum(1 for row in sub_data if row[f_idx] == val)
27         # Using Laplace smoothing
28         self.conditional[c][f_idx][val] = (count + 1) / (sub_total +
29 len(set(feature_vals)))
30     def predict(self, sample):
31         best_class = None
32         best_prob = -1.0
33
34         for c in self.classes:
35             prob = self.prior[c]
36             for f_idx, val in enumerate(sample):
37                 if val in self.conditional[c][f_idx]:
38                     prob *= self.conditional[c][f_idx][val]
39             if prob > best_prob:
40                 best_prob = prob
41                 best_class = c
42         return best_class
43     # Example Usage
44     if __name__ == "__main__":
45         dataset = [
46             ['Sunny', 'High', 'No'],
```

```
3 # Example Usage
4 if __name__ == "__main__":
5     dataset = [
6         ['Sunny', 'High', 'No'],
7         ['Sunny', 'High', 'No'],
8         ['Overcast', 'High', 'Yes'],
9         ['Rainy', 'Normal', 'Yes'],
10        ['Rainy', 'Normal', 'Yes']
11    ]
12
13    nb = NaiveBayes()
14    nb.fit(dataset)
15    test_instance = ['Sunny', 'Normal']
16    print(f"Predicted class for {test_instance}:", nb.predict(test_instance))
17
18
```

Program 9: Naive Bayes Classification (Second Run)

Assigned Labels: [0 0 0 1 1 1]

Calculated Silhouette Validation Score: 0.7133477791749615

Zainab khatoun 24becs173